Backend Developer Candidate Task

Objective

Build a secure, RESTful backend service for a **Gold Trading System**. The system should allow users to register, deposit money, buy/sell gold, and withdraw money.

Requirements

Core Features

1. **User Management**
   - Register and Login using email and password.
   - Store:
     - user_id (UUID)
     - name (String)
     - email (String, with validation and uniqueness)
     - password (hashed)
     - role (admin/user)
   - Authentication: Use JWT, include user role in the payload.
2. **Wallet Management**
   - Users can:
     - Deposit money (simulate via API).
     - View balance.
     - Withdraw money (simulate via API with validation).
   - Balance cannot go negative.
   - Maintain transaction history with:
     - transaction_id (UUID)
     - user_id
     - type: deposit/withdraw/trade
     - amount
     - timestamp
3. **Gold Trading**
   - Users can:
     - Buy gold using their money balance.
     - Sell gold to convert back to money.
   - Track:
     - trade_id (UUID)
     - user_id
     - type: buy/sell
     - gold_amount (grams)
     - price_per_gram (mocked or static)
     - total_cost

    - timestamp
  - Constraints:
    - Cannot buy if balance is insufficient.
    - Cannot sell more than owned gold.
    - Store current gold holding for each user.
4. Admin Panel
   - Admins can:
     - View all users and their balances.
     - View all transactions.
     - Adjust balances (optional advanced feature).

Advanced Requirements

1. **Search and Filters**

   - Search transactions by:
     - user name
     - transaction type
     - date range

2. **Rate Limiting**

   - Protect routes like login, deposit, withdrawal.

3. **Pagination**

   - For transaction and trading logs.

4. **Soft Delete**

   - Users can be deactivated (soft delete). - Deleted users cannot log in or transact.

5. **Security**
   - Use RBAC to separate admin and user access.
   - Validate all inputs (avoid SQL injection/XSS).
   - Passwords hashed using bcrypt or equivalent.

Technical Constraints

- Backend Framework: Node.js (TypeScript, Express/NestJS) or Go (Gin/Fiber)
- Database: PostgreSQL or MySQL using ORM (TypeORM, Sequelize, GORM)
- Authentication: JWT with role-based payload
- Docker: Containerize backend using docker

Deliverables

- Push to a private GitHub repo.
- Share access with jackkaphon.
- Use meaningful commits.
- Include all endpoints with sample requests/responses.

Evaluation Criteria

1. **Code Quality:**
   - Clean, readable, and well-documented code.
   - Use of proper design patterns.
2. **Database Design:**
   - Proper normalization and use of indexes for performance.
3. **API Design:**
   - RESTful standards
   - Intuitive and consistent endpoint design
4. **Advanced Features:**
   - Effective implementation of search, pagination, rate limiting, and soft delete.
5. **Security:**
   - Robust authentication and validation mechanisms.
6. **Dockerization:**
   - Smooth setup and running of the application using Docker.

Bonus

- Simulate real-time gold price via a background service or mock API.
- Admin-initiated system-wide broadcast messages.
- Support for multi-currency (USD, LAK, etc.)

Note

We do not expect perfection across all fields. Instead, we want to evaluate your skills in structuring and implementing logic, as well as how you approach problem-solving. You are free to be flexible and use techniques or tools you are most familiar with, as long as they meet the task's objectives. Feel free to include any additional features or improvements you believe would enhance the system.